

방해자:

동시 스레드 간 데이터 교환을 위한 제한된 대기열에 대한 고성능 대안

마틴 톰슨
데이브 팔리
마이클 바커
패트리샤 지
앤드류 스튜어트

2011년 5월

<http://code.google.com/p/disruptor/>

1 초록

LMAX는 매우 높은 성과를 내는 금융 거래소를 만들기 위해 설립되었습니다. 이 목표를 달성하기 위한 노력의 일환으로 저희는 이러한 시스템 설계에 대한 여러 접근 방식을 평가해 왔습니다. 하지만 측정을 시작하면서 기존 접근 방식의 근본적인 한계에 부딪혔습니다.

많은 애플리케이션이 처리 단계 간 데이터 교환을 위해 큐에 의존합니다. 성능 테스트 결과, 이러한 방식으로 큐를 사용할 때 발생하는 지연 시간은 RAID 또는 SSD 기반 디스크 시스템의 디스크 입출력 작업 비용과 거의 같은 수준으로, 매우 느린 것으로 나타났습니다. 엔드투엔드 작업에 여러 큐가 있는 경우 전체 지연 시간에 수백 마이크로초가 추가됩니다. 분명히 최적화의 여지가 있습니다.

추가 조사와 컴퓨터 과학에 대한 집중을 통해 기존 접근 방식(예: 대기열과 처리 노드)에 내재된 문제가 합쳐지면 멀티스레드 구현에서 경합이 발생한다는 사실을 깨달았고, 더 나은 접근 방식이 있을 수 있음을 시사했습니다.

현대 CPU가 작동하는 방식, 즉 우리가 "기계적 공감"이라고 부르는 것에 대해 생각해 보고, 우리 사향을 분리하는 데 중점을 둔 우수한 설계 관행을 사용하여 Disruptor라고 명명한 데이터 구조와 사용 패턴을 생각해 냈습니다.

테스트 결과, 3단계 파이프라인에서 Disruptor를 사용한 평균 지연 시간은 동일한 큐 기반 방식보다 100배 더 짧은 것으로 나타났습니다. 또한, Disruptor는 동일한 구성에서 약 8배 더 많은 처리량을 처리합니다.

이러한 성능 향상은 동시 프로그래밍에 대한 사고방식의 획기적인 변화를 의미합니다. 이 새로운 패턴은 높은 처리량과 낮은 지연 시간이 요구되는 모든 비동기 이벤트 처리 아키텍처에 이상적인 기반이 될 것입니다.

LMAX는 이러한 패턴을 기반으로 주문 매칭 엔진, 실시간 위험 관리, 그리고 고가용성 인메모리 거래 처리 시스템을 성공적으로 구축했습니다. 각 시스템은 저희가 아는 한 타의 추종을 불허하는 새로운 성능 기준을 제시했습니다.

하지만 이는 금융 업계에만 적용되는 전문 솔루션이 아닙니다. Disruptor는 동시 프로그래밍에서 복잡한 문제를 성능을 극대화하는 방식으로 해결하고 구현이 간편한 범용 메커니즘입니다. 일부 개념이 생소하게 보일 수 있지만, 저희 경험상 이 패턴으로 구축된 시스템은 다른 유사한 메커니즘보다 구현이 훨씬 간단합니다.

Disruptor는 쓰기 경합이 현저히 적고, 동시성 오버헤드가 낮으며, 다른 방식보다 캐시 친화적입니다. 이 모든 것이 더 낮은 지연 시간에서 지터가 적고 처리량이 더 높습니다. 적당한 클럭 속도의 프로세서에서 초당 2,500만 개 이상의 메시지를 처리하고 지연 시간이 50나노초 미만인 것을 확인했습니다. 이러한 성능은 지금까지 본 다른 구현 방식과 비교해 상당한 개선입니다. 이는 최신 프로세서가 코어 간에 데이터를 교환할 수 있는 이론적 한계에 매우 가깝습니다.

2 개요

Disruptor는 LMAX에서 세계 최고 성능의 금융 거래소를 구축하고자 노력한 결과입니다. 초기 설계는 파이프라인을 통해 처리량을 높이는 SEDA1 과 Actors2 에서 파생된 아키텍처에 중점을 두었습니다. 다양한 구현 사례를 프로파일링한 결과, 파이프라인 단계 간 이벤트 큐잉이 비용의 대부분을 차지한다는 것이 분명해졌습니다.

큐가 지연 시간과 높은 수준의 지터를 발생시킨다는 것을 발견했습니다. 더 나은 성능을 가진 새로운 큐 구현을 개발하는 데 상당한 노력을 기울였습니다. 그러나 프로듀서, 컨슈머, 그리고 데이터 저장에 대한 설계 고려 사항이 뒤섞여 큐가 기본 데이터 구조로서 한계를 지닌다는 것이 분명해졌습니다. Disruptor는 이러한 고려 사항을 명확하게 분리하는 동시성 구조를 구축하려는 노력의 결과입니다.

3 동시성의 복잡성

이 문서와 컴퓨터 과학 전반의 맥락에서 동시성은 두 개 이상의 작업이 병렬로 수행되는 것뿐만 아니라, 리소스 접근을 놓고 경쟁하는 것을 의미합니다. 경쟁하는 리소스는 데이터베이스, 파일, 소켓 또는 메모리의 위치일 수 있습니다.

코드의 동시 실행은 상호 배제와 변경 사항의 가시성, 이 두 가지와 관련이 있습니다. 상호 배제는 특정 리소스에 대한 경쟁되는 업데이트를 관리하는 것입니다.

변경 사항의 가시성은 그러한 변경 사항이 언제 수행되는지 제어하는 것입니다.

다른 스레드에서 볼 수 있습니다. 경쟁 업데이트의 필요성을 없앨 수 있다면 상호 배제의 필요성을 피할 수 있습니다. 알고리즘이 주어진 리소스가 단 하나의 스레드에 의해서만 수정되도록 보장할 수 있다면 상호 배제는 불필요합니다. 읽기 및 쓰기 작업은 모든 변경 사항을 다른 스레드에서 볼 수 있도록 요구합니다. 하지만

경합되는 쓰기 작업에는 변경 사항의 상호 배제가 필요합니다.

모든 동시 환경에서 가장 비용이 많이 드는 작업은 경합된 쓰기 접근입니다. 여러 스레드가 동일한 리소스에 쓰기 작업을 수행하려면 복잡하고 비용이 많이 드는 조정이 필요합니다. 일반적으로 이는 일종의 잠금 전략을 사용하여 달성됩니다.

3.1 잠금 비용

잠금은 상호 배제를 제공하고 변경 사항의 가시성이 순서대로 발생하도록 보장합니다. 잠금은 경합 시 중재가 필요하기 때문에 비용이 매우 많이 듭니다. 이 중재는 운영 체제 커널로의 컨텍스트 전환을 통해 이루어지며, 잠금이 해제될 때까지 잠금을 기다리는 스레드를 일시 중단합니다. 이러한 컨텍스트 전환 과정에서 운영 체제는 제어권을 반환하고, 제어권을 유지하는 동안 다른 관리 작업을 수행하기로 결정할 수 있습니다. 실행 컨텍스트는 이전에 캐시된 데이터와 명령어를 잃을 수 있습니다. 이는 최신 프로세서의 성능에 심각한 영향을 미칠 수 있습니다. 빠른 사용자 모드 잠금을 사용할 수 있지만, 경합이 발생하지 않을 때만 실질적인 이점을 얻을 수 있습니다.

간단한 데모를 통해 잠금의 비용을 살펴보겠습니다. 이 실험의 초점은 루프에서 64비트 카운터를 5억 번 증가시키는 함수를 호출하는 것입니다. 이 함수는 Java로 작성하면 2.4Ghz Intel Westmere EP에서 단일 스레드로 단 300ms 만에 실행할 수 있습니다. 이 실험에서 언어는 중요하지 않으며, 동일한 기본 요소를 사용하는 모든 언어에서 결과는 유사할 것입니다.

상호 배제를 제공하기 위해 잠금을 도입하면, 잠금이 아직 경합되지 않았더라도 비용이 상당히 증가합니다. 두 개 이상의 스레드가 경합하기 시작하면 비용은 다시 몇 배나 증가합니다. 이 간단한 실험의 결과는 아래 표에 나와 있습니다.

방법	시간(ms)
단일 스레드	300
잠금 장치가 있는 단일 스레드	10,000
잠금 장치가 있는 두 개의 스레드	22만 4천
CAS가 있는 단일 스레드	5,700
CAS가 있는 두 개의 스레드	3만
휘발성 쓰기가 있는 단일 스레드	4,700

표 1 - 경쟁의 비교 비용

3.2 “CAS” 비용

업데이트 대상이 단일 단어일 때 메모리를 업데이트할 때 잠금 사용보다 더 효율적인 대안을 사용할 수 있습니다. 이러한 대안은 최신 버전에 구현된 원자적 또는 연동 명령어를 기반으로 합니다.

프로세서. 이러한 작업은 일반적으로 CAS(Compare And Swap) 작업으로 알려져 있으며, 예를 들어 x86에서는 "lock cmpxchg"와 같습니다. CAS 연산은 메모리에 있는 단어를 원자 연산으로 조건부로 설정할 수 있도록 하는 특수 기계어 코드 명령어입니다. "카운터 증가 실패"의 경우, 각 스레드는 카운터를 읽은 후 원자적으로 증가된 값으로 설정하려고 시도하는 루프를 돌 수 있습니다. 이전 값과 새 값은 이 명령어의 매개변수로 제공됩니다. 만약,

작업이 실행될 때 카운터 값이 제공된 예상 값과 일치하면 카운터가 새 값으로 업데이트됩니다. 반면, 값이 예상과 다르면 CAS 작업이 실패합니다. 그러면 변경을 시도하는 스레드가 해당 값부터 증가하여 카운터를 다시 읽는 작업을 변경이 완료될 때까지 반복해야 합니다.

성공합니다. 이 CAS 방식은 중재를 위해 커널로의 컨텍스트 전환이 필요하지 않기 때문에 잠금 방식보다 훨씬 효율적입니다. 하지만 CAS 연산은 무료가 아닙니다. 프로세서는 원자성을 보장하기 위해 명령어 파이프라인을 잠그고, 메모리 배리어를 사용하여 변경 사항을 다른 스레드에 공개해야 합니다. CAS 연산은 Java에서 `java.util.concurrent.Atomic*` 클래스를 사용하여 사용할 수 있습니다.

프로그램의 중요 섹션이 단순한 카운터 증가보다 더 복잡한 경우, 경합을 조율하기 위해 여러 CAS 연산을 사용하는 복잡한 상태 머신이 필요할 수 있습니다. 잠금을 사용하여 동시성 프로그램을 개발하는 것은 어렵습니다. CAS 연산과 메모리 배리어를 사용하여 잠금 없는 알고리즘을 개발하는 것은 훨씬 더 복잡하며, 그 알고리즘이 정확하다는 것을 증명하는 것도 매우 어렵습니다.

이상적인 알고리즘은 단일 리소스에 대한 모든 쓰기 작업을 단일 스레드가 담당하고 다른 스레드들이 결과를 읽는 방식입니다. 다중 프로세서 환경에서 결과를 읽으려면 다른 프로세서에서 실행 중인 스레드에 변경 사항을 표시할 수 있도록 메모리 배리어가 필요합니다.

3.3 기억 장벽

최신 프로세서는 성능상의 이유로 명령어를 비순차적으로 실행하고, 메모리와 실행 유닛 간에 데이터를 비순차적으로 로드 및 저장합니다. 프로세서는 프로그램 로직이 실행 순서와 관계없이 동일한 결과를 생성하도록 보장하면 됩니다. 이는 단일 스레드 프로그램에서는 문제가 되지 않습니다. 그러나 스레드가 상태를 공유하는 경우, 데이터 교환이 성공적으로 이루어지려면 모든 메모리 변경 사항이 필요한 시점에 순서대로 나타나는 것이 중요합니다. 메모리 배리어는 프로세서가 메모리 순서를 변경해야 하는 코드 섹션을 나타내는 데 사용됩니다.

업데이트는 중요합니다. 이는 스레드 간에 하드웨어 순서 지정 및 변경 사항 가시성을 확보하는 수단입니다. 컴파일러는 컴파일된 코드의 순서를 보장하기 위해 보편적인 소프트웨어 장벽을 적용할 수 있으며, 이러한 소프트웨어 메모리 장벽은 프로세서 자체에서 사용하는 하드웨어 장벽 외에도 추가로 적용됩니다.

최신 CPU는 현재 세대의 메모리 시스템보다 훨씬 빠릅니다. 이러한 차이를 메우기 위해 CPU는 체이닝 없이 사실상 빠른 하드웨어 해시 테이블인 복잡한 캐시 시스템을 사용합니다. 이러한 캐시는 메시지 전달 프로토콜을 통해 다른 프로세서 캐시 시스템과 일관성을 유지합니다. 또한, 프로세서에는 "저장 버퍼"가 있습니다.

이러한 캐시에 쓰기 작업을 오프로드하고 "대기열을 무효화"하여 캐시 일관성 프로토콜이 쓰기가 발생하기 직전에 무효화 메시지를 신속하게 확인하여 효율성을 높일 수 있습니다.

데이터에 있어 이것이 의미하는 바는, 어떤 값의 최신 버전이 기록된 후 어느 단계에서든 레지스터, 저장 버퍼, 여러 캐시 계층 중 하나 또는 주 메모리에 존재할 수 있다는 것입니다. 스레드가 이 값을 공유하려면, 순서대로 가시화되어야 하며, 이는 캐시 일관성 메시지의 조정된 교환을 통해 달성됩니다. 이러한 메시지의 적시 생성은 메모리 배리어를 통해 제어될 수 있습니다.

읽기 메모리 배리어는 캐시에 들어오는 변경 사항에 대해 무효화 대기열의 한 지점을 표시하여 CPU에 로드 명령을 순서대로 전달합니다. 이를 통해 읽기 배리어보다 먼저 처리된 쓰기 작업에 대한 일관된 뷰를 제공합니다.

쓰기 배리어는 저장 버퍼에 특정 지점을 표시하여 CPU에 저장 명령을 전달하고, 이를 통해 캐시를 통해 쓰기 작업을 플러시합니다. 이 배리어는 쓰기 배리어 이전에 어떤 저장 작업이 수행되는지에 대한 순서화된 시각을 제공합니다.

전체 메모리 장벽은 로드와 저장을 모두 명령하지만 이를 실행하는 CPU에만 명령합니다.

일부 CPU에는 이 세 가지 기본 요소 외에도 더 많은 변형이 있지만, 이 세 가지만으로도 관련된 복잡성을 이해하기에 충분합니다. Java 메모리 모델에서 휘발성 필드의 읽기 및 쓰기는 읽기 및 쓰기를 구현합니다.

각각 쓰기 장벽을 사용합니다. 이는 Java 5 출시와 함께 정의된 Java 메모리 모델3에 명시되었습니다.

3.4 캐시 라인

최신 프로세서에서 캐싱이 사용되는 방식은 성공적인 고성능 운영에 매우 중요합니다. 이러한 프로세서는 캐시에 저장된 데이터와 명령어를 처리하는 데는 매우 효율적이지만, 캐시 미스가 발생하면 상대적으로 비효율적입니다.

저희 하드웨어는 메모리를 바이트나 워드 단위로 이동하지 않습니다. 효율성을 위해 캐시는 일반적으로 32~256바이트 크기의 캐시 라인으로 구성되며, 가장 일반적인 캐시 라인은 64바이트입니다. 이는 캐시 일관성 프로토콜이 작동하는 세분성 수준입니다. 즉, 두 변수가 동일한 캐시 라인에 있고 서로 다른 스레드에서 데이터를 쓰는 경우, 마치 단일 변수인 경우와 동일한 쓰기 경합 문제가 발생합니다. 이는 "가짓 공유"라는 개념입니다. 따라서 고성능을 위해서는 경합을 최소화하기 위해 독립적이지만 동시에 작성된 변수들이 동일한 캐시 라인을 공유하지 않도록 하는 것이 중요합니다.

예측 가능한 방식으로 메모리에 액세스할 때, CPU는 다음에 액세스될 가능성이 높은 메모리를 예측하고 백그라운드에서 캐시에 미리 가져오는 방식으로 메인 메모리 액세스의 지연 시간 비용을 숨길 수 있습니다. 이는 프로세서가 예측 가능한 "스트라이드"를 사용하여 메모리를 이동하는 것과 같은 액세스 패턴을 감지할 수 있는 경우에만 작동합니다. 배열의 내용을 반복할 때 스트라이드는 예측 가능하므로 캐시 라인에서 메모리를 미리 가져오므로 액세스 효율성이 극대화됩니다. 일반적으로 프로세서가 인식하려면 스트라이드가 양방향으로 2048바이트 미만이어야 합니다. 그러나 연결 리스트나 트리와 같은 데이터 구조는 메모리에 노드가 더 넓게 분산되어 있고 액세스 스트라이드가 예측 가능하지 않은 경향이 있습니다. 메모리에 일관된 패턴이 없으면 시스템이 캐시 라인을 미리 가져오는 능력이 제한되어 메인 메모리 액세스의 효율성이 두 자릿수 이상 떨어질 수 있습니다.

3.5 대기열의 문제점

큐는 일반적으로 연결 리스트나 배열을 기본 요소 저장 공간으로 사용합니다. 메모리 내 큐가 무제한으로 허용될 경우, 여러 유형의 문제에서 큐가 검사되지 않고 커져 치명적인 오류 지점에 도달할 수 있습니다.

메모리를 고갈시키는 것입니다. 이는 생산자가 소비자보다 속도가 빠를 때 발생합니다. 무제한 큐는 생산자가 소비자보다 속도가 빠르지 않고 메모리가 귀중한 자원인 시스템에서 유용할 수 있지만, 이 가정이 성립하지 않고 큐가 무제한으로 커지면 항상 위험이 따릅니다. 이러한 치명적인 결과를 방지하기 위해 큐는 일반적으로 크기가 제한됩니다(제한됨). 큐를 제한하려면 배열 기반이거나 크기를 적극적으로 추적해야 합니다.

큐 구현은 헤드, 테일, 크기 변수에 쓰기 경합이 발생하는 경향이 있습니다. 사용 시, 큐는 일반적으로 소비자와 생산자 간의 속도 차이로 인해 항상 거의 가득 차거나 거의 비어 있습니다. 생산 속도와 소비 속도가 균등하게 일치하는 균형 잡힌 중간 지점에서 작동하는 경우는 매우 드뭅니다. 이러한 항상 가득 차거나 항상 비어 있는 경향은 높은 수준의 경합 및/또는 캐시 일관성에 대한 비용 부담을 초래합니다. 문제는 헤드와 테일 메커니즘이 잠금이나 CAS 변수와 같은 서로 다른 동시 객체를 사용하여 분리되더라도 일반적으로 동일한 캐시 라인을 차지한다는 것입니다.

큐의 헤드를 차지하는 프로듀서, 테일을 차지하는 컨슈머, 그리고 그 사이의 노드 저장을 관리하는 문제는 큐에 단일 라지 그레인 잠금을 사용하는 것 이상으로 동시 구현 설계를 매우 복잡하게 만듭니다. 풋(put) 및 테이크(take) 연산을 위해 전체 큐에 라지 그레인 잠금을 적용하는 것은 구현하기는 간단하지만 처리량에 심각한 병목 현상을 초래합니다. 이러한 동시 구현 문제를 큐의 의미 체계 내에서 분리하면, 단일 프로듀서-단일 컨슈머 구현이 아닌 다른 구현의 경우 구현이 매우 복잡해집니다.

Java에서는 큐를 사용하는 데 또 다른 문제가 있습니다. 큐가 심각한 가비지 소스이기 때문입니다. 첫째, 객체를 할당하고 큐에 넣어야 합니다. 둘째, 연결 리스트를 사용하는 경우 객체를 할당하여 목록의 노드입니다. 더 이상 참조되지 않으면 대기열 구현을 지원하기 위해 할당된 모든 객체를 다시 회수해야 합니다.

3.6 파이프라인 및 그래프

많은 종류의 문제에 대해 여러 처리 단계를 파이프라인으로 연결하는 것이 합리적입니다. 이러한 파이프라인은 종종 병렬 경로를 가지며 그래프와 같은 토폴로지로 구성됩니다. 각 단계 간의 연결은 종종 다음과 같습니다. 각 단계마다 고유한 스레드가 있는 대기열로 구현됩니다.

이 접근 방식은 비용이 많이 듭니다. 각 단계에서 작업 단위를 큐에 추가하고 큐에서 제거하는 데 비용이 발생합니다. 경로가 분기되어야 할 경우 대상의 수는 이 비용을 곱하고, 분기 후 다시 연결해야 할 경우 불가피하게 경합 비용이 발생합니다.

단계 사이에 대기열을 두는 데 드는 비용을 들이지 않고도 종속성 그래프를 표현할 수 있다면 이상적일 것입니다.

4 LMAX 파괴기의 설계

위에서 설명한 문제들을 해결하려고 시도하는 동안, 큐에 얽혀 있다고 생각했던 문제들을 엄격하게 분리하는 설계가 등장했습니다. 이 접근 방식은 모든 데이터가 쓰기 접근을 위해 단 하나의 스레드에만 속하도록 하는 데 중점을 두어 쓰기 경합을 제거하는 데 중점을 두었습니다. 이 설계는 "Disruptor"로 알려지게 되었습니다. 종속성 그래프를 처리하는 방식이 Java 7에서 Fork-Join을 지원하기 위해 도입된 "Phasers" 4 개념과 유사하기 때문에 이러한 이름이 붙었습니다.

LMAX 디스rupter는 위에서 설명한 모든 문제를 해결하여 메모리 할당의 효율성을 극대화하고 캐시 친화적인 방식으로 작동하여 최신 하드웨어에서 최적의 성능을 발휘하도록 설계되었습니다.

디스rupter 메커니즘의 핵심은 링 버퍼 형태의 미리 할당된 제한된 데이터 구조입니다. 데이터는 하나 이상의 생산자를 통해 링 버퍼에 추가되고 하나 이상의 소비자에 의해 처리됩니다.

4.1 메모리 할당

링 버퍼의 모든 메모리는 시작 시 미리 할당됩니다. 링 버퍼는 항목에 대한 포인터 배열이나 항목을 나타내는 구조체 배열을 저장할 수 있습니다. Java 언어의 한계로 인해 항목은 객체에 대한 포인터로 링 버퍼와 연결됩니다. 이러한 각 항목은 일반적으로 전달되는 데이터 자체가 아니라 해당 데이터를 저장하는 컨테이너입니다. 이러한 항목의 사전 할당은 가비지 컬렉션을 지원하는 언어에서 발생하는 문제를 해결합니다. 항목이 재사용되고 Disruptor 인스턴스가 지속되는 동안 유지되기 때문입니다. 이러한 항목의 메모리는 동시에 할당되며, 주 메모리에 연속적으로 배치될 가능성이 매우 높아 캐시 스트라이딩을 지원합니다. John Rose는 Java 언어에 "값 유형"을 도입하자는 제안을 했습니다. 이 제안은 C와 같은 다른 언어처럼 튜플 배열을 허용하여 메모리가 연속적으로 할당되고 포인터 간접 참조를 방지합니다.

Java와 같은 관리형 런타임 환경에서 저지연 시스템을 개발할 때 가비지 컬렉션은 문제가 될 수 있습니다. 할당되는 메모리가 많을수록 가비지 컬렉터의 부담이 커집니다. 가비지 컬렉터는 객체의 수명이 매우 짧거나 사실상 영구적일 때 가장 효과적으로 작동합니다. 링 버퍼에 항목을 미리 할당한다는 것은 가비지 컬렉터 입장에서는 영구적이므로 부담이 적다는 것을 의미합니다.

과부하 시 대기열 기반 시스템이 백업될 수 있으며 이로 인해 처리 속도가 감소하고 결과가 발생할 수 있습니다.

할당된 객체가 예상보다 오래 지속되어 젊은 세대를 넘어 승격된

세대별 가비지 컬렉터. 이는 두 가지 의미를 갖습니다. 첫째, 객체를 세대 간에 복사해야 하므로 지연 시간 지연이 발생합니다. 둘째, 이러한 객체를 이전 세대에서 수집해야 하는데, 이는 일반적으로 훨씬 더 많은 비용이 드는 작업이며, 단편화된 메모리 공간을 압축해야 할 때 발생하는 "Stop the World" 일시 중지 가능성을 높입니다. 대용량 메모리 힙에서는 이로 인해 GB당 몇 초씩 일시 중지가 발생할 수 있습니다.

4.2 우려 사항 분석

우리는 다음과 같은 문제가 모든 대기열 구현에서 합쳐지는 것을 보았습니다. 즉, 이러한 별개의 동작의 집합이 대기열이 구현하는 인터페이스를 정의하는 경향이 있다는 것입니다.

1. 교환되는 품목의 보관
2. 교환을 위한 다음 시퀀스를 청구하는 생산자의 조정
3. 새로운 품목이 출시되었다는 알림을 소비자에게 전달

가비지 컬렉션을 사용하는 언어로 금융 거래소를 설계할 때, 과도한 메모리 할당은 문제가 될 수 있습니다. 따라서 앞서 설명했듯이 연결 리스트 기반 큐는 좋은 접근 방식이 아닙니다. 처리 단계 간 데이터 교환을 위한 전체 저장 공간을 미리 할당할 수 있다면 가비지 컬렉션은 최소화됩니다. 또한, 이러한 할당이 균일한 청크로 수행될 수 있다면, 해당 데이터의 순회는 최신 프로세서에서 사용하는 캐싱 전략에 매우 친화적인 방식으로 수행될 것입니다. 이러한 요구 사항을 충족하는 데이터 구조는 모든 슬롯이 미리 채워진 배열입니다. 링 버퍼를 생성할 때, Disruptor는 추상 팩토리 패턴을 사용하여 항목을 미리 할당합니다. 항목이 요청되면 프로듀서는 해당 데이터를 미리 할당된 구조에 복사할 수 있습니다.

대부분의 프로세서에서는 링의 슬롯을 결정하는 시퀀스 번호의 나머지 계산에 매우 높은 비용이 발생합니다. 링 크기를 2의 거듭제곱으로 만들면 이 비용을 크게 줄일 수 있습니다. 크기에서 1을 뺀 비트 마스크를 사용하면 나머지 연산을 효율적으로 수행할 수 있습니다.

앞서 설명했듯이, 유계 큐는 큐의 헤드와 테일에서 경합을 겪습니다. 링 버퍼 자료 구조는 이러한 경합 및 동시성 기본 요소에서 자유롭습니다. 링 버퍼에 접근하기 위해 생산자 및 소비자 장벽을 통해 이러한 문제가 해결되었기 때문입니다. 이러한 장벽의 논리는 아래에 설명되어 있습니다.

Disruptor의 가장 일반적인 사용 사례에서는 프로듀서가 하나뿐입니다. 일반적인 프로듀서는 파일 리더 또는 네트워크 리스너입니다. 프로듀서가 하나인 경우 시퀀스/엔트리 할당에 대한 경합이 발생하지 않습니다.

여러 생산자가 있는 좀 더 특이한 경우, 생산자들은 링 버퍼의 다음 엔트리를 차지하기 위해 서로 경쟁합니다. 다음 사용 가능한 엔트리를 차지하기 위한 경쟁은 해당 슬롯의 시퀀스 번호에 대한 간단한 CAS 연산을 통해 관리할 수 있습니다.

프로듀서가 관련 데이터를 클레임된 항목에 복사하면 시퀀스를 커밋하여 컨슈머에게 공개할 수 있습니다. 이는 CAS 없이도 다른 프로듀서들이 각자의 커밋에서 해당 시퀀스에 도달할 때까지 간단한 비지 스피ن(busy spin)을 통해 수행할 수 있습니다. 그런 다음, 해당 프로듀서는 커서를 앞으로 이동하여 다음으로 사용 가능한 항목을 표시할 수 있습니다.

생산자는 링 버퍼에 쓰기 전에 간단한 읽기 작업으로 소비자의 순서를 추적하여 링을 래핑하는 것을 피할 수 있습니다.

소비자는 항목을 읽기 전에 링 버퍼에서 시퀀스가 사용 가능해질 때까지 기다립니다. 대기하는 동안 다양한 전략을 사용할 수 있습니다. CPU 리소스가 부족한 경우 생산자가 신호를 보내는 잠금 내의 조건 변수를 기다릴 수 있습니다. 이는 분명히 경합 지점이며 CPU 리소스가 지연 시간이나 처리량보다 더 중요할 때만 사용해야 합니다. 소비자는 링 버퍼에서 현재 사용 가능한 시퀀스를 나타내는 커서를 반복하여 확인할 수도 있습니다. 이는 CPU 리소스와 지연 시간을 교환하여 스레드 수율을 사용하거나 사용하지 않고 수행할 수 있습니다. 잠금 및 조건 변수를 사용하지 않으면 생산자와 소비자 간의 경합 종속성이 끊어지므로 확장성이 매우 뛰어납니다. 잠금 없는 다중 생산자 - 다중 소비자 큐는 존재하지만 헤드, 테일, 크기 카운터에 대한 여러 CAS 작업이 필요합니다. Disruptor는 이러한 CAS 경합을 겪지 않습니다.

4.3 시퀀싱

시퀀싱은 Disruptor에서 동시성을 관리하는 방식의 핵심 개념입니다. 각 프로듀서와 컨슈머는 링 버퍼와 상호 작용하는 방식에 대해 엄격한 시퀀싱 개념을 적용합니다. 프로듀서는 링에서 항목을 요청할 때 시퀀스의 다음 슬롯을 요청합니다. 다음으로 사용 가능한 슬롯의 시퀀스는 프로듀서가 하나뿐인 경우 간단한 카운터이거나, 여러 프로듀서인 경우 CAS 연산을 사용하여 업데이트되는 원자 카운터일 수 있습니다. 시퀀스 값이 요청되면 링 버퍼의 이 항목은 이제 요청하는 프로듀서가 쓸 수 있게 됩니다. 프로듀서가 항목 업데이트를 완료하면 컨슈머가 사용할 수 있는 최신 항목에 대한 링 버퍼의 커서를 나타내는 별도의 카운터를 업데이트하여 변경 사항을 커밋할 수 있습니다. 링 버퍼 커서는 아래와 같이 CAS 연산을 요구하지 않고도 프로듀서가 메모리 배리어를 사용하여 바쁜 스피ن에서 읽고 쓸 수 있습니다.

```
긴 예상 시퀀스 = 주장된 시퀀스 - 1;
while (커서 != expectedSequence)
{
    // 바쁜 스피ن
}
```

```
커서 = 주장된 시퀀스;
```

소비자는 메모리 배리어를 사용하여 커서를 읽음으로써 주어진 시퀀스가 사용 가능해질 때까지 기다립니다. 커서가 업데이트되면 메모리 배리어는 커서가 진행될 때까지 기다린 소비자에게 링 버퍼 항목의 변경 사항을 표시합니다.

각 소비자는 링 버퍼의 항목을 처리하면서 업데이트되는 고유한 시퀀스를 포함합니다. 이러한 소비자 시퀀스를 통해 생산자는 링이 래핑되는 것을 방지하기 위해 소비자를 추적할 수 있습니다. 또한 소비자 시퀀스를 통해 소비자는 동일한 항목에 대한 작업을 순서대로 조정할 수 있습니다.

생산자가 하나뿐인 경우, 소비자 그래프의 복잡성과 관계없이 잠금이나 CAS 연산이 필요하지 않습니다. 논의된 메모리 배리어만으로 전체 동시성 조정을 달성할 수 있습니다.

시퀀스.

4.4 배치 효과

소비자가 링 버퍼에서 커서가 이동하는 시퀀스를 기다릴 때, 큐에서는 불가능한 흥미로운 상황이 발생합니다. 소비자가 링 버퍼 커서가 마지막으로 이동한 이후 여러 단계 이동했음을 발견하면,

확인 결과, 동시성 메커니즘에 관여하지 않고 해당 시퀀스까지 처리할 수 있습니다. 이로 인해 생산자가 앞서 나가면 뒤처진 소비자도 생산자와 빠르게 속도를 맞춰 시스템의 균형을 이룬다. 이러한 유형의 배치는 처리량을 증가시키는 동시에 지연 시간을 줄이고 완화합니다. 저희 관찰 결과에 따르면, 이 효과는 메모리 하위 시스템까지 부하에 관계없이 지연 시간이 거의 일정하게 유지되는 결과를 낳습니다. 포화 상태이면 프로파일은 리틀의 법칙6에 따라 선형적입니다. 이는 대기열에서 부하가 증가함에 따라 관찰된 지연 시간에 대한 "J" 곡선 효과와는 매우 다릅니다.

4.5 종속성 그래프

큐는 생산자와 소비자 간의 단순한 1단계 파이프라인 종속성을 나타냅니다. 소비자가 종속성의 사슬 또는 그래프와 같은 구조를 형성하면 그래프의 각 단계 사이에 큐가 필요합니다. 이로 인해 종속 단계 그래프 내에서 큐의 고정 비용이 여러 번 발생합니다. LMAX 금융 거래소를 설계할 때, 프로파일링 결과 큐 기반 접근 방식을 사용했을 때 거래 처리에 드는 총 실행 비용 중 큐 비용이 차지하는 비율이 더 컸습니다.

Disruptor 패턴을 사용하면 프로듀서와 컨슈머의 관심사가 분리되므로, 코어에 단일 링 버퍼만 사용하면서도 컨슈머 간의 복잡한 종속성 그래프를 표현할 수 있습니다. 이를 통해 고정 실행 비용이 크게 절감되어 처리량이 증가하고 지연 시간은 단축됩니다.

단일 링 버퍼를 사용하면 전체 워크플로를 나타내는 복잡한 구조의 항목을 응집된 공간에 저장할 수 있습니다. 이러한 구조를 설계할 때는 개별 사용자가 작성한 상태로 인해 캐시 라인의 잘못된 공유가 발생하지 않도록 주의해야 합니다.

4.6 Disruptor 클래스 다이어그램

Disruptor 프레임워크의 핵심 관계는 아래 클래스 다이어그램에 나와 있습니다. 이 다이어그램에서는 프로그래밍 모델을 단순화하는 데 사용할 수 있는 편의 클래스는 제외되어 있습니다. 종속성 그래프가 구축되면 프로그래밍 모델은 단순해집니다. 프로듀서는 `ProducerBarrier`를 통해 항목을 순서대로 클레임하고, 클레임된 항목에 변경 사항을 작성한 다음, `ProducerBarrier`를 통해 해당 항목을 커밋하여 소비할 수 있도록 합니다. 소비자는 새 항목이 사용 가능할 때 콜백을 수신하는 `BatchHandler` 구현만 제공하면 됩니다. 이렇게 생성된 프로그래밍 모델은 이벤트 기반이며, Actor 모델과 많은 유사점을 갖습니다.

큐 구현에서 일반적으로 결합되는 문제를 분리하면 더 유연한 설계가 가능합니다. `RingBuffer`

Disruptor 패턴의 핵심으로, 경합 없이 데이터 교환을 위한 저장소를 제공합니다. `RingBuffer`와 상호 작용하는 프로듀서와 컨슈머의 동시성 문제는 분리됩니다. `ProducerBarrier`

링 버퍼에서 슬롯을 클레임할 때 발생하는 동시성 문제를 관리하는 동시에, 링이 래핑되는 것을 방지하기 위해 종속된 소비자를 추적합니다. `ConsumerBarrier`는 새로운 항목이 있을 때 소비자에게 알리고, `Consumer`는 처리 파이프라인의 여러 단계를 나타내는 종속성 그래프로 구성될 수 있습니다.

```
// 소비자가 구현할 수 있는 콜백 핸들러
최종 BatchHandler<ValueEntry> 배치 핸들러 = 새 BatchHandler<ValueEntry>()
{
    public void onAvailable(final ValueEntry entry)는 예외를 throw합니다.
    {
        // 새로운 항목이 생기면 처리합니다.
    }

    public void onEndOfBatch()는 예외를 발생시킵니다.
    {
        // 필요한 경우 결과를 IO 장치로 플러싱하는 데 유용합니다.
    }

    공개 무효 onCompletion()
    {
        // 종료하기 전에 필요한 정리 작업을 수행합니다.
    }
};

RingBuffer<ValueEntry> ringBuffer =
    새로운 RingBuffer<ValueEntry>(ValueEntry.ENTRY_FACTORY, SIZE,
                                   ClaimStrategy.Option.SINGLE_THREADED,
                                   대기전략을선택양보);
ConsumerBarrier<ValueEntry> consumerBarrier = ringBuffer.createConsumerBarrier(); BatchConsumer<ValueEntry> batchConsumer =
    새로운 BatchConsumer<ValueEntry>(consumerBarrier, batchHandler);
생산자 배리어<ValueEntry> 생산자 배리어 = ringBuffer.createProducerBarrier(배치 소비자);

// 각 소비자는 별도의 스레드에서 실행될 수 있습니다.
EXECUTOR.submit(배치 소비자);
```



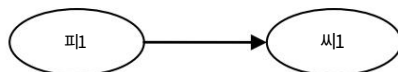
```
// 생산자는 순서대로 항목을 청구합니다.
ValueEntry 항목 = producerBarrier.nextEntry();

// 데이터를 엔트리 컨테이너에 복사합니다.

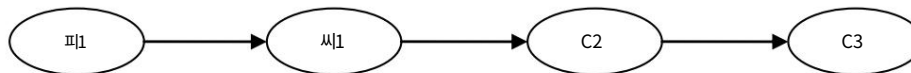
// 소비자에게 항목을 제공합니다
producerBarrier.commit(항목);
```

5 처리량 성능 테스트

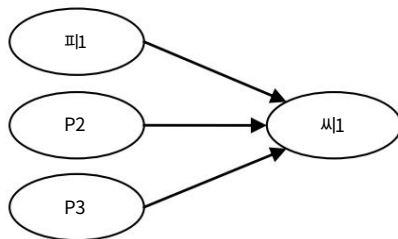
참고로, Doug Lea의 훌륭한 `java.util.concurrent.ArrayBlockingQueue7`을 선택했습니다. 이 큐는 테스트 결과 모든 바운디드 큐 중 가장 높은 성능을 보였습니다. 테스트는 Disruptor와 동일한 블로킹 프로그래밍 스타일로 진행됩니다. 아래에 자세히 설명된 테스트 사례는 Disruptor 오픈 소스 프로젝트에서 확인할 수 있습니다. 참고: 테스트를 실행하려면 최소 4개의 스레드를 병렬로 실행할 수 있는 시스템이 필요합니다.



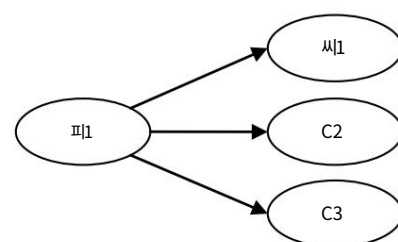
유니캐스트: 1P - 1C



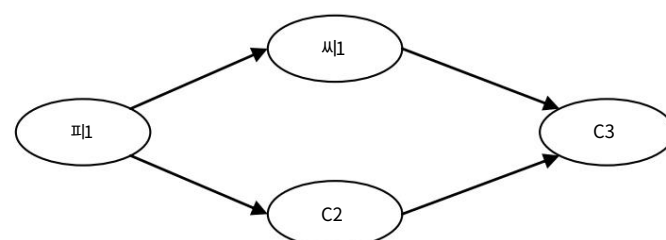
3단계 파이프라인: 1P - 3C



시퀀서: 3P - 1C



멀티캐스트: 1P - 3C



다이아몬드: 1P - 3C

위 구성에서는 각 데이터 흐름 아크에 ArrayBlockingQueue를 적용했으며, Disruptor를 사용한 배리어 구성과 비교했습니다. 다음 표는 Java 1.6.0_25 64비트 Sun JVM, Windows 7, Intel Core i7 860 @ 2.8GHz(HT 미적용), Intel Core i7-2720QM, Ubuntu 11.04를 사용하여 초당 연산 수(OPS)로 측정된 성능 결과를 보여줍니다.

5억 개의 메시지를 처리할 때 3번의 실행 중 가장 좋은 결과를 얻었습니다. 결과는 JVM 실행 방식에 따라 크게 다를 수 있으며, 아래 수치는 저하가 관찰한 가장 높은 값이 아닙니다.

	Nehalem 2.8Ghz - Windows 7 SP1 64비트		샌디 브릿지 2.2Ghz - 리눅스 2.6.38 64비트	
	ABQ	디스ruptor	ABQ	디스ruptor
유니캐스트: 1P - 1C	5,339,256	25,998,336	4,057,453	22,381,378
파이프라인: 1P - 3C	2,128,918	16,806,157	2,006,903	15,857,913
시퀀서: 3P - 1C	5,539,531	13,403,268	2,056,118	14,540,519
멀티캐스트: 1P - 3C	1,077,384	9,377,871	260,733	10,860,121
다이아몬드: 1P - 3C	2,113,941	16,143,613	2,082,725	15,295,197

표 2 - 비교 처리량(초당 작업 수)

6 지연 성능 테스트

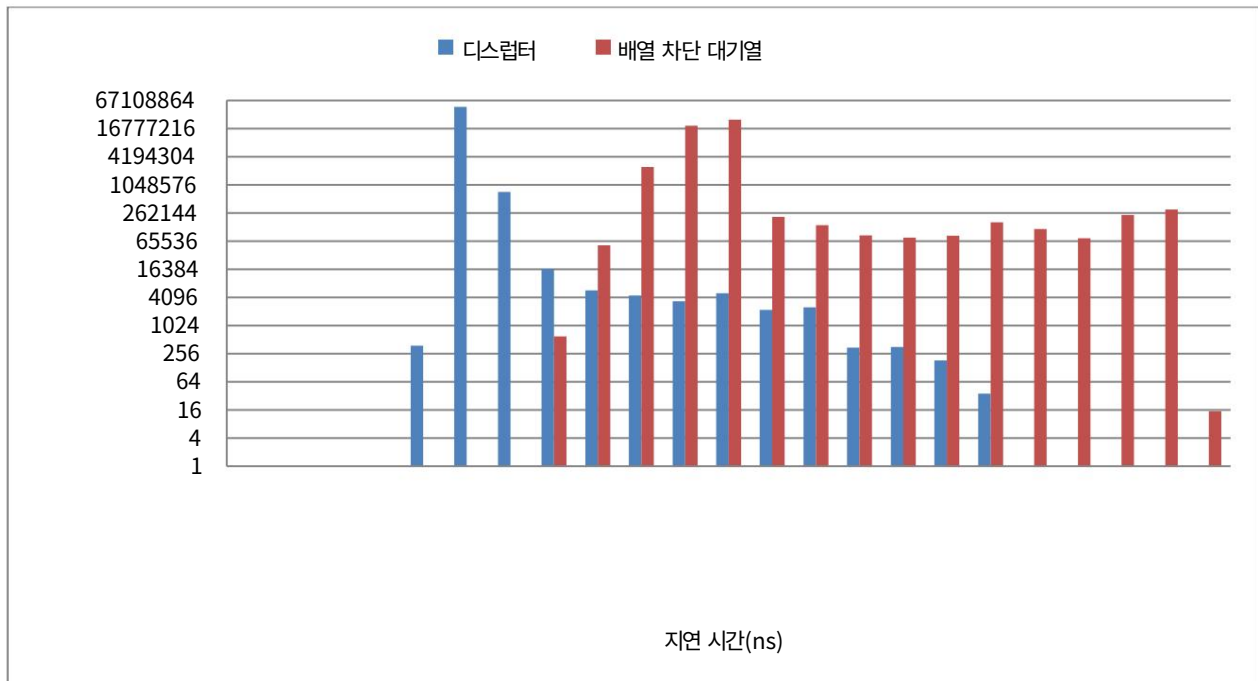
지연 시간을 측정하기 위해 3단계 파이프라인을 사용하여 포화 상태보다 낮은 이벤트들을 생성합니다. 이는 이벤트 주입 후 1마이크로초 동안 기다린 후 다음 이벤트를 주입하고 5천만 번 반복함으로써 달성됩니다. 이러한 정밀도로 시간을 측정하려면 CPU의 타임스탬프 카운터를 사용해야 합니다. 구형 프로세서는 절전 및 절전 모드로 인해 주파수가 변하기 때문에 불변 TSC를 사용하는 CPU를 선택했습니다. Intel Nehalem 및 이후 프로세서는 Ubuntu 11.04에서 실행되는 최신 Oracle JVM에서 액세스할 수 있는 불변 TSC를 사용합니다. 이 테스트에는 CPU 바인딩이 사용되지 않았습니다.

비교를 위해 ArrayBlockingQueue를 다시 한번 사용해 보겠습니다. 더 나은 결과를 얻을 수 있는 ConcurrentLinkedQueue8을 사용할 수도 있었지만, 생산자가 소비자보다 앞서서 백프레서를 발생시키지 않도록 제한된 큐 구현을 사용하고자 합니다. 아래 결과는 Ubuntu 11.04에서 Java 1.6.0_25 64비트를 실행하는 2.2Ghz Core i7-2720QM의 결과입니다.

Disruptor의 홀당 평균 지연 시간은 52나노초인 반면, ArrayBlockingQueue의 지연 시간은 32,757나노초입니다. 프로파일링 결과, 잠금 및 조건 변수를 통한 신호 전달이 ArrayBlockingQueue 지연 시간의 주요 원인임을 알 수 있습니다.

	배열 차단 대기열(ns)	디스ruptor(ns)
최소 지연 시간	145	29
평균 지연 시간	32,757	52
99% 관찰치 미만	2,097,152	128
99.99% 관찰치 미만	4,194,304	8,192
최대 지연 시간	5,069,086	175,567

표 3 - 3단계 파이프라인의 비교 지연 시간



7 결론

Disruptor는 동시 실행 컨텍스트 간 대기 시간을 줄이고 처리량을 증가시키는 데 있어 중요한 단계입니다.

예측 가능한 지연 시간을 보장하는 것은 많은 애플리케이션에서 중요한 고려 사항입니다. 저희 테스트 결과, 스레드 간 데이터 교환에 있어 동급의 다른 접근 방식보다 성능이 뛰어납니다. 저희는 이것이 이러한 데이터 교환에 있어 가장 높은 성능을 내는 메커니즘이라고 생각합니다. 스레드 간 데이터 교환과 관련된 문제를 명확하게 분리하고, 쓰기 경합을 제거하고, 읽기 경합을 최소화하며, 최신 프로세서에서 사용되는 캐싱과 코드가 원활하게 작동하도록 보장함으로써, 저희는 모든 애플리케이션에서 스레드 간 데이터 교환을 위한 매우 효율적인 메커니즘을 개발했습니다.

소비자가 주어진 임계값까지 경합 없이 항목을 처리할 수 있도록 하는 배치 효과는 고성능 시스템에 새로운 특징을 도입합니다. 대부분의 시스템에서 부하와 경합이 증가함에 따라 지연 시간이 기하급수적으로 증가하는 특징적인 "J" 곡선이 나타납니다. Disruptor의 부하가 증가함에 따라 지연 시간은 메모리 하위 시스템이 포화 상태에 도달할 때까지 거의 일정하게 유지됩니다.

우리는 Disruptor가 고성능 컴퓨팅의 새로운 기준을 제시했으며 프로세서와 컴퓨터 설계의 현재 추세를 계속 활용할 수 있는 매우 좋은 위치에 있다고 믿습니다.

¹ 단계적 이벤트 기반 아키텍처 - <http://www.eecs.harvard.edu/~mdw/proj/seda/>

² 액터 모델 - <http://dspace.mit.edu/handle/1721.1/6952>

³ 자바 메모리 모델 - <http://www.ibm.com/developerworks/library/j-jtp02244/index.html>

⁴ 페이지 - <http://gee.cs.oswego.edu/dl/jsr166/dist/jsr166ydocs/jsr166y/Phaser.html>

⁵ 값 유형 - http://blogs.oracle.com/jrose/entry/tuples_in_the_vm

⁶ 리틀의 법칙 - http://en.wikipedia.org/wiki/Little%27s_law

⁷ ArrayBlockingQueue - <http://download.oracle.com/javase/1.5.0/docs/api/java/util/concurrent/ArrayBlockingQueue.html>

⁸ ConcurrentLinkedQueue - <http://download.oracle.com/javase/1.5.0/docs/api/java/util/concurrent/ConcurrentLinkedQueue.html>